

# SCHOOL OF COMPUTER SCIENCE AND IT YBN UNIVERSITY RANCHI

## JAVA PROGRAMMING ASSIGNMENT 11

### Problem 1: Calculate Factorial Using Recursion

#### Description:

Write a Java program that calculates the factorial of a given number using recursion. The program should prompt the user to input a positive integer and then use a recursive method named `factorial` to compute the factorial of that number. The method should return the factorial value, which will be printed in the `main` method.

#### Concepts Covered:

- Recursion
- Halting condition (base case)

#### Hint:

- The factorial of a number `n` is `n * factorial(n - 1)` with the base case `factorial(0) = 1`.
- Ensure that the recursive method has a base case to stop the recursion.

### Problem 2: Recursive Sum of Digits

#### Description:

Write a Java program that takes an integer as input and recursively calculates the sum of its digits. For example, if the input is 1234, the program should return 10 (because  $1 + 2 + 3 + 4 = 10$ ). The program should include a method named `sumOfDigits` that performs this calculation using recursion.

#### Concepts Covered:

- Recursion
- Halting condition (base case)

#### Hint:

- The sum of the digits of a number `n` can be found by `sumOfDigits(n) = n % 10 + sumOfDigits(n / 10)`, with the base case `sumOfDigits(0) = 0`.

### Problem 3: Recursive Fibonacci Sequence

#### Description:

Write a Java program that calculates the `n`th Fibonacci number using recursion. The program should prompt the user to input a positive integer `n` and then use a recursive method named

`fibonacci` to compute and return the `n`th Fibonacci number. The Fibonacci sequence starts with 0, 1, 1, 2, 3, 5, 8..., where each number is the sum of the two preceding ones.

**Concepts Covered:**

- Recursion
- Halting condition (base case)

**Hint:**

- The `n`th Fibonacci number can be found by `fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)` with the base cases `fibonacci(0) = 0` and `fibonacci(1) = 1`.
- Ensure the method has the base cases to prevent infinite recursion.